Project Title
Smart Hospital Bed & Operation Room Scheduling System
Project Report submitted for the partial fulfilment of grade for the course
Data Structures and Algorithms -3 (25CS2103E)
in
A.Y. 2026-2027
Submitted by
A. Shashank Reddy - 2520090139
CH. Sai Vihamsh - 2520090090
B. Dharshan - 2520090088
Team Number: 26
Section: 11
Under the guidance of
Mrs. Ch. Anitha
Asst. Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled "**Smart Hospital Bed & Operation Room Scheduling System**" is a record of Bonafide work of **A. Shashank Reddy – 2520090139, CH. Sai Vihamsh – 2520090090, B. Dharshan – 2520090088** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

A. Shashank Reddy – 2520090139
CH. Sai Vihamsh – 2520090090
B. Dharshan – 2520090088

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Smart Hospital Bed & Operation Room Scheduling System**" is a bonafide work done and submitted by **A. Shashank Reddy – 2520090139, CH. Sai Vihamsh – 2520090090, B. Dharshan – 2520090088** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success of this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our guide, Mrs. Ch. Anitha, Assistant Professor, Department of Computer Science and Information Technology, whose tremendous support, encouragement and guidance made it possible to complete this project.

We express our gratitude to **Dr. Khaja Shareef**, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project-based Lab.

We thank all the members of the teaching and non-teaching staff, and also those who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

A. Shashank Reddy – 2520090139

CH. Sai Vihamsh – 2520090090

B. Dharshan – 2520090088

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11

S.No.	Contents	Page no.
6.	Implementation	12
7.	Results & Discussion	20
8.	Conclusion & Future Scope	23
9.	References	24

1. ABSTRACT

The Hospital Bed and Operating Room (OR) Scheduling System is an automated hospital management solution designed to improve the efficient utilisation of hospital beds, operating rooms, and medical staff. The system assigns available beds to patients and schedules surgeries by considering bed availability, operating room availability, doctor schedules, surgery duration, and patient priority. Emergency cases are given higher priority to ensure timely treatment and reduce delays in critical situations.

The system replaces manual scheduling with an automated approach, reducing errors, double bookings, scheduling conflicts, and unnecessary patient waiting time. It also helps hospital administrators monitor and utilise available resources effectively. By coordinating patients, doctors, beds, and operating rooms through a centralised system, the hospital can improve workflow and make faster scheduling decisions.

The system was built in Java from first principles using data structures studied in the course: a binary heap serves patients by priority, hash maps and queues allocate and release beds in constant time, and ordered maps (red-black trees) hold the timetable of every operating room and doctor so that a clash is detected in logarithmic time. The report derives the time and space complexity of each operation. In a test with ten patients, eight beds, two operating rooms and three surgeons, all emergency patients were served first, no room or doctor was double-booked, and beds freed by discharges were reassigned to waiting patients in priority order.

Team Name: Team 26

Team Members:

1. A. Shashank Reddy_2520090139
2. CH. Sai Vihamsh_2520090090
3. B. Dharshan_2520090088

2. INTRODUCTION

2.1 Background

A hospital has a limited number of beds, operating rooms and surgeons, while the demand for them is uneven. Planned (elective) operations can be arranged in advance, but emergency patients arrive without warning and cannot wait behind routine cases. In many hospitals the allocation is still done by phone calls, registers and whiteboards, which leads to double bookings, idle rooms, last-minute clashes and long waiting times. A surgery needs several resources at the same time: an operating room, a surgeon and, after admission, a bed in the right ward. If any one of them is missing, the whole plan fails. Planning of operating theatres and related resources has therefore been studied for many years [2], [3], [4].

2.2 Motivation

A scheduling decision must be quick and must never create a conflict. Each part of this problem is served by a different data structure: serving the most urgent patient first needs a priority queue, finding a patient's bed needs fast hashing, and checking whether a time slot is already taken needs an ordered search structure. The project was chosen because it brings together, in one realistic system, several topics of the Data Structures and Algorithms course: heaps, hashing, queues, balanced search trees (ordered maps) and greedy algorithms. The outcome is also easy to verify by hand, because a schedule is either conflict-free or it is not.

2.3 Proposed Approach

This project builds a Hospital Bed and Operating Room Scheduling System in Java using only the standard library. Patients are registered with a priority, a ward, and, if needed, a surgery duration and surgeon. The system then performs the following tasks:

- **Priority-based triage** using a binary min-heap ordered by priority (emergency, urgent, routine) and arrival time, so that the most urgent patient is always served next.
- **Bed allocation** using a hash map from ward to a queue of free beds, so that a bed is allocated or released in constant time, with a waiting queue for each ward when it is full.
- **Operating room and doctor scheduling** using one ordered map (red-black tree) per room and per doctor, so that overlapping bookings are rejected and the earliest free slot is found.
- **Discharge handling and monitoring** which frees the bed, admits the highest-priority waiting patient of that ward, and reports bed occupancy and operating room utilisation.

The report is organised as follows: Chapter 3 reviews related work, Chapter 4 states the problem and objectives, Chapter 5 lists the requirements, Chapter 6 explains the implementation together with its complexity analysis, Chapter 7 presents the results, Chapter 8 gives the conclusion and future scope, and Chapter 9 lists the references.

3. LITERATURE SURVEY

Scheduling of hospital resources has been studied in operations research and in algorithm design. The following works and approaches were studied while designing this system.

3.1 Priority queues and heaps [1], [5]. Williams introduced the binary heap as the structure behind heapsort, and Cormen et al. describe how the same structure implements a priority queue in which insertion and removal of the highest-priority element take O(log n) time. Our triage queue uses exactly this structure, ordered by patient priority and then by arrival time.

3.2 Interval scheduling [1], [7]. Kleinberg and Tardos treat the scheduling of requests that need a shared resource for a period of time, and show that greedy strategies work well for such interval problems. An operating room or a surgeon is such a resource, a booking is an interval, and two bookings conflict when their intervals overlap. Our earliest-free-slot search follows the same greedy idea.

3.3 Balanced search trees and ordered maps [1], [6], [8]. Guibas and Sedgewick introduced red-black trees, which keep keys sorted and support insertion and neighbour look-ups in O(log n) time. The Java class TreeMap is built on a red-black tree [8], and its floorEntry and ceilingKey methods let us find the booking just before or just after a time in logarithmic time, which is all that is needed to test an overlap.

3.4 Operating room planning and scheduling [2]. Cardoen, Demeulemeester and Beliën review the literature on operating room planning and scheduling and classify it by the patients considered, the performance measures used, the decisions taken and the solution techniques applied. They also discuss the other limited resources, such as surgeons and beds, that restrict an operating room schedule, which is the setting of our system.

3.5 Operational research in the operating theatre [3]. Guerriero and Guido survey operational-research methods for managing the operating theatre at the strategic, tactical and operational levels. Many of the methods are based on mathematical programming or heuristics. Our system works at the operational level, where decisions must be taken quickly as patients arrive and are discharged.

3.6 Planning decisions in health care [4]. Hulshof et al. give a taxonomy of planning decisions in health care, including capacity planning of resources and the admission of patients. Their classification covers both ward and operating theatre planning, which supports handling bed management and operating room scheduling together in one system.

3.7 Manual and first-come-first-served scheduling. The simplest practice is to keep the schedule in a register or on a whiteboard and to serve patients in the order of arrival. It needs no software, but nothing checks for clashes, and an emergency patient can end up waiting behind routine cases. It is the baseline that the proposed system improves on.

3.8 Comparison of Existing Approaches with the Proposed System

Table 3.1 compares the approaches studied above with the proposed system.

Approach	Core technique	Priority handling	Conflict detection	Limitation
Manual scheduling	Registers, phone calls	Staff judgement	Manual checking	Errors, double bookings, slow updates
First-come-first-served	Plain FIFO queue	None	None built in	Emergencies wait behind routine cases
Mathematical programming [2], [3]	Integer programming and heuristics	Weights in the objective	Constraints in the model	Needs a solver; hard to build and explain
Proposed system	Heap, hash maps, ordered maps	Emergency, urgent, routine; FCFS for ties	O(log n) overlap test per room and doctor	In-memory prototype without a user interface

Table 3.1: Comparison of existing approaches with the proposed system

3.9 Summary and Research Gap

The literature offers detailed optimisation models for operating room planning, and textbooks explain priority queues, ordered maps and interval scheduling separately. What a student needs is a small, readable program in which these structures work together on one realistic problem. The present project fills this gap by combining a priority heap, hash maps, queues and ordered interval maps into one Java system for admission, bed allocation and operating room scheduling with conflict detection, and by justifying the time and space complexity of every operation.

4. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement: Hospitals schedule beds, operating rooms and surgeons manually, which causes double bookings, scheduling conflicts, idle resources and long waiting times, especially for emergency patients who must not wait behind routine cases.

Problem Description: A scheduling system must accept patients of different priorities, give each a bed in a suitable ward, find for every surgery the earliest slot at which an operating room and the surgeon are both free, reject any booking that overlaps an existing one, and give freed beds to waiting patients, all with algorithms whose complexity is justified.

Objectives:

- To design and build an automated bed and operating room scheduling system in Java without using any external library.
- To serve patients by priority, with emergency cases first and arrival time deciding between equal priorities, using a binary heap.
- To allocate and release beds ward by ward in constant time, and to keep a waiting queue for each ward when it is full.
- To schedule surgeries in the earliest slot at which both an operating room and the surgeon are free, and to reject any double booking.
- To reassign freed beds to waiting patients automatically when a patient is discharged, and to report bed occupancy and operating room utilisation.
- To analyse and justify the time and space complexity of the admission, bed management and scheduling operations.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 / AMD Ryzen 3 / Apple M-series or higher (dual-core, 2.0 GHz or above)
- RAM: 4 GB minimum (8 GB recommended)
- Storage: at least 500 MB of free disk space for the project and the tools
- Display: 1366 × 768 resolution or higher

Software Requirements:

- Operating System: macOS, Windows 10 / 11 or Linux
- Programming Language: Java (JDK 17 or later) for the scheduling engine; the code in this report was compiled and run on JDK 21
- Data: patient, ward, bed, operating room and doctor details, defined in the program as test input
- IDE / Tools: IntelliJ IDEA or VS Code, with Git for version control
- Libraries: only the Java standard library (java.util); no external library is used

6. IMPLEMENTATION

6.1 System Architecture and Workflow

The system works as a pipeline. Patients are first added to a triage heap. In the admission phase the most urgent patient is removed from the heap and a bed is requested from the free-bed queue of the required ward. If no bed is free, the patient is placed in the waiting queue of that ward. If a bed is found and the patient needs surgery, the scheduler searches all operating rooms for the earliest slot at which the room and the surgeon are both free, and books it. When a patient is discharged, the bed is released and the highest-priority waiting patient of that ward is admitted at once.

Patient requests	→	Priority queue (heap)	→	Bed allocation (hash map + queues)	→	OR scheduler (ordered maps)	→	Schedule and bed report

Fig. 6.1: Processing pipeline of the scheduling system

- **Triage queue:** a binary heap that orders patients by priority and then by arrival time.
- **Bed manager:** keeps the free beds of each ward in a queue and records which patient occupies which bed.
- **OR scheduler:** keeps one ordered timetable per operating room and per doctor, rejects overlaps and finds the earliest free slot.
- **Waiting queues and driver:** one heap per ward for patients who could not get a bed, and the main program that handles admission, discharge and reporting.

6.2 Data Structures and Algorithms Used

Component	Data structure / Algorithm	Purpose	Complexity
Triage queue	Binary min-heap ordered by (priority, arrival)	Serve the most urgent patient first	Insert and remove O(log P)
Waiting queues	TreeMap of ward → heap	Hold patients when a ward is full	O(log W) per operation
Bed pool	HashMap of ward → ArrayDeque of free beds	Allocate and release beds	O(1) average
Occupancy records	HashMap bed → patient and patient → bed	Find a patient's bed on discharge	O(1) average
OR timetable	TreeMap (red-black tree) start → booking, per room	Overlap test and ordered timetable	O(log B) per test
Doctor timetable	TreeMap start → booking, per doctor	Prevent a doctor being double-booked	O(log Bd) per test
Slot search	Greedy earliest feasible start over candidate times (TreeSet)	Choose room and start time	O(R·(B+Bd)·log(B+Bd))

Table 6.1: Data structures and algorithms

6.3 Module 1: Patient and Triage Queue (Patient.java, PatientQueue.java)

A patient is stored as an immutable record with an identifier, name, priority (1 = emergency, 2 = urgent, 3 = routine), ward, surgery duration in minutes, surgeon and arrival number. The triage queue is a binary min-heap stored in an array. Two patients are compared by priority first and by arrival number second, so emergency patients always come out first and patients of equal priority are served in the order they arrived. Insertion sifts the new patient up, and removal moves the last patient to the root and sifts it down, each in O(log P) time.

```
public record Patient(int id, String name, int priority, String ward,
                      int surgeryMins, String doctor, int arrival) {
    // priority: 1 = Emergency, 2 = Urgent, 3 = Routine
    public String label() { return "P" + id + " " + name; }
}


import java.util.*;


public final class PatientQueue {
    private Patient[] heap = new Patient[16];
    private int size;
```

```
// Emergency first; ties are broken by earlier arrival (first come, first served).
private static boolean before(Patient a, Patient b) {
    return a.priority() != b.priority() ? a.priority() < b.priority()
                                        : a.arrival() < b.arrival();
}

public void add(Patient p) {                        // O(log n)
    if (size == heap.length) heap = Arrays.copyOf(heap, size * 2);
    heap[size] = p;
    siftUp(size++);
}

public Patient poll() {                             // O(log n)
    if (size == 0) return null;
    Patient top = heap[0];
    heap[0] = heap[--size];
    heap[size] = null;
    if (size > 0) siftDown(0);
    return top;
}

public boolean isEmpty() { return size == 0; }
public int size() { return size; }

private void siftUp(int i) {
    while (i > 0) {
        int p = (i - 1) / 2;
        if (!before(heap[i], heap[p])) break;
        swap(i, p); i = p;
    }
}
```

```
    private void siftDown(int i) {
        while (true) {
            int l = 2 * i + 1, r = l + 1, m = i;
            if (l < size && before(heap[l], heap[m])) m = l;
            if (r < size && before(heap[r], heap[m])) m = r;
            if (m == i) break;
            swap(i, m); i = m;
        }
    }

    private void swap(int a, int b) { Patient t = heap[a]; heap[a] = heap[b]; heap[b] = t; }
}
```

6.4 Module 2: Bed Management (BedManager.java)

The bed manager keeps, for each ward, a queue of free beds. Allocating a bed takes the first bed of the queue, and releasing a bed puts it back at the end, so both are O(1). Two hash maps record which patient lies in which bed and the reverse, so the bed of a discharged patient is found without any search. If the ward has no free bed, allocation returns null and the caller places the patient in the waiting queue.

```
import java.util.*;

public final class BedManager {
    private final Map<String, Deque<String>> freeBeds = new HashMap<>();   // ward -> free beds
    private final Map<String, Integer> occupant = new HashMap<>();         // bed  -> patient id
    private final Map<Integer, String> bedOf = new HashMap<>();            // patient id -> bed
    private int totalBeds;

    public void addWard(String ward, int count) {
        Deque<String> q = new ArrayDeque<>();
        for (int i = 1; i <= count; i++) q.add(ward + "-" + i);
        freeBeds.put(ward, q);
        totalBeds += count;
```

```
    }

    /** Gives the next free bed of the ward to the patient: O(1). Returns null if the ward is
full. */
    public String allocate(String ward, int patientId) {
        Deque<String> q = freeBeds.get(ward);
        if (q == null || q.isEmpty()) return null;
        String bed = q.poll();
        occupant.put(bed, patientId);
        bedOf.put(patientId, bed);
        return bed;
    }

    /** Discharges the patient and returns the bed to its ward: O(1). */
    public String release(int patientId) {
        String bed = bedOf.remove(patientId);
        if (bed == null) return null;
        occupant.remove(bed);
        freeBeds.get(bed.substring(0, bed.indexOf('-'))).addLast(bed);
        return bed;
    }

    public int freeCount(String ward) { return freeBeds.get(ward).size(); }
    public int occupied() { return occupant.size(); }
    public int total() { return totalBeds; }
}
```

6.5 Module 3: Operating Room and Doctor Scheduling (OperatingRoomScheduler.java)

Each operating room and each doctor has a TreeMap that maps the start time of a booking to the booking. To test whether the interval [s, e) is free, the scheduler looks at the last booking that starts at or before s (floorEntry) and checks that it ends by s, and then looks at the first booking that starts at or after s (ceilingKey) and checks that it starts at or after e. Because stored bookings never overlap, these two look-ups are enough, and the test takes O(log B) time. A booking is accepted only when both the room and the doctor are free, which prevents double booking.

To find the earliest slot, the scheduler uses the fact that a surgery can start either at the earliest allowed time or at the moment a booking of the room or the doctor ends. For every room it collects these candidate times in a sorted set, tests them in increasing order, and keeps the earliest feasible start over all rooms. Time is counted in minutes from 08:00.

```
import java.util.*;


public final class OperatingRoomScheduler {

    public record Booking(int start, int end, int patientId, String room, String doctor) {}


    private final Map<String, TreeMap<Integer, Booking>> roomBookings = new TreeMap<>();

    private final Map<String, TreeMap<Integer, Booking>> doctorBookings = new HashMap<>();


    public OperatingRoomScheduler(String... rooms) {

        for (String r : rooms) roomBookings.put(r, new TreeMap<>());

    }


    /** True if [s, e) overlaps no booking in the map: two O(log n) tree look-ups. */

    private static boolean isFree(TreeMap<Integer, Booking> m, int s, int e) {

        Map.Entry<Integer, Booking> prev = m.floorEntry(s);      // last booking starting at or
before s

        if (prev != null && prev.getValue().end() > s) return false;

        Integer next = m.ceilingKey(s);                           // first booking starting at or
after s

        return next == null || next >= e;

    }


    /** Books a fixed slot. Rejects it if the room or the doctor is already busy. */

    public boolean book(String room, int s, int e, int patientId, String doctor) {
```

```
        TreeMap<Integer, Booking> rm = roomBookings.get(room);

        TreeMap<Integer, Booking> dm = doctorBookings.computeIfAbsent(doctor, k -> new
TreeMap<>());

        if (!isFree(rm, s, e) || !isFree(dm, s, e)) return false;

        Booking b = new Booking(s, e, patientId, room, doctor);

        rm.put(s, b);

        dm.put(s, b);

        return true;

    }



    /** Finds the earliest slot (any room) at which room and doctor are both free, and books it.
*/

    public Booking schedule(int patientId, String doctor, int duration, int earliest) {

        TreeMap<Integer, Booking> dm = doctorBookings.computeIfAbsent(doctor, k -> new
TreeMap<>());

        Booking best = null;

        for (Map.Entry<String, TreeMap<Integer, Booking>> re : roomBookings.entrySet()) {

            TreeSet<Integer> candidates = new TreeSet<>();        // a slot can only start when
something ends

            candidates.add(earliest);

            for (Booking b : re.getValue().values()) if (b.end() > earliest)
candidates.add(b.end());

            for (Booking b : dm.values())            if (b.end() > earliest)
candidates.add(b.end());

            for (int t : candidates) {

                if (best != null && t >= best.start()) break;     // cannot beat the best slot
found so far

                if (isFree(re.getValue(), t, t + duration) && isFree(dm, t, t + duration)) {

                    best = new Booking(t, t + duration, patientId, re.getKey(), doctor);

                    break;

                }

            }

        }

        if (best != null) book(best.room(), best.start(), best.end(), patientId, doctor);

        return best;

    }
```

```
    public Map<String, TreeMap<Integer, Booking>> rooms() { return roomBookings; }


    public double utilisation(String room, int window) {

        int busy = 0;

        for (Booking b : roomBookings.get(room).values()) busy += b.end() - b.start();

        return 100.0 * busy / window;

    }

}
```

6.6 Module 4: Admission, Waiting Queue and Discharge (HospitalScheduler.java)

The driver program creates the wards and operating rooms, registers ten patients in the triage heap, and admits them in priority order. A patient who cannot get a bed goes into the waiting heap of the ward. When a patient is discharged, the bed is released and the first patient of the waiting heap of that ward, which is the one with the highest priority and the earliest arrival, is admitted and, if needed, scheduled for surgery from the current time. Finally the program prints the bed occupancy and the utilisation of each operating room.

```
import java.util.*;


public final class HospitalScheduler {

    static final String[] PRIORITY = {"", "EMERGENCY", "URGENT", "ROUTINE"};

    static final BedManager beds = new BedManager();

    static final OperatingRoomScheduler ors = new OperatingRoomScheduler("OR1", "OR2");

    static final Map<String, PatientQueue> waiting = new TreeMap<>();   // one waiting queue per
ward


    static String clock(int t) { return String.format("%02d:%02d", 8 + t / 60, t % 60); }  // day
starts 08:00


    static void admit(Patient p, int now) {

        String bed = beds.allocate(p.ward(), p.id());

        if (bed == null) {

            waiting.computeIfAbsent(p.ward(), k -> new PatientQueue()).add(p);

            System.out.printf("    %-11s %-10s -> no free %s bed, added to waiting queue%n",
```

```
                    PRIORITY[p.priority()], p.label(), p.ward());
            return;
        }
        System.out.printf("    %-11s %-10s -> bed %s%n", PRIORITY[p.priority()], p.label(), bed);
        if (p.surgeryMins() > 0) {
            OperatingRoomScheduler.Booking b = ors.schedule(p.id(), p.doctor(), p.surgeryMins(),
now);
            System.out.printf("                              surgery %s-%s in %s with %s%n",
                    clock(b.start()), clock(b.end()), b.room(), p.doctor());
        }
    }

    static void discharge(Patient p, int now) {
        String bed = beds.release(p.id());
        System.out.printf("    %s: %s discharged, bed %s freed%n", clock(now), p.label(), bed);
        PatientQueue q = waiting.get(p.ward());
        if (q != null && !q.isEmpty()) admit(q.poll(), now);
    }

    public static void main(String[] args) {
        beds.addWard("ICU", 2);
        beds.addWard("SURGICAL", 3);
        beds.addWard("GENERAL", 3);

        Patient[] ps = {
            new Patient(1,  "Ravi",   3, "GENERAL",    0, null,       0),
            new Patient(2,  "Anita",  3, "SURGICAL",  90, "Dr. Rao",  1),
            new Patient(3,  "Kiran",  1, "ICU",      120, "Dr. Rao",  2),
            new Patient(4,  "Meena",  2, "SURGICAL",  60, "Dr. Sen",  3),
            new Patient(5,  "Suresh", 3, "SURGICAL",  45, "Dr. Khan", 4),
            new Patient(6,  "Lata",   1, "ICU",      150, "Dr. Sen",  5),
            new Patient(7,  "Farhan", 2, "GENERAL",    0, null,       6),
            new Patient(8,  "Divya",  3, "SURGICAL",  60, "Dr. Rao",  7),
```

```
            new Patient(9,  "Harsh",  1, "ICU",        0, null,       8),
            new Patient(10, "Neha",   2, "SURGICAL",  75, "Dr. Khan", 9)};

        PatientQueue queue = new PatientQueue();
        for (Patient p : ps) queue.add(p);

        System.out.println("[1] Admission in priority order (beds + surgeries, day starts
08:00)");
        while (!queue.isEmpty()) admit(queue.poll(), 0);

        System.out.println("\n[2] Operating-room timetable");
        for (var e : ors.rooms().entrySet()) {
            StringBuilder sb = new StringBuilder("    " + e.getKey() + ": ");
            for (var b : e.getValue().values())
                sb.append(String.format("%s-%s P%d (%s)  ", clock(b.start()), clock(b.end()),
b.patientId(), b.doctor()));
            System.out.println(sb.toString().stripTrailing());
        }

        System.out.println("\n[3] Conflict detection");
        System.out.println("    Book OR1 09:00-10:00 for Dr. Khan  -> " + (ors.book("OR1", 60,
120, 11, "Dr. Khan") ? "ACCEPTED" : "REJECTED (room busy)"));
        System.out.println("    Book OR2 12:00-13:00 for Dr. Rao   -> " + (ors.book("OR2", 240,
300, 12, "Dr. Rao") ? "ACCEPTED" : "REJECTED (doctor busy)"));
        System.out.println("    Book OR2 14:00-15:00 for Dr. Sen   -> " + (ors.book("OR2", 360,
420, 13, "Dr. Sen") ? "ACCEPTED" : "REJECTED"));

        System.out.println("\n[4] Discharges free beds for the waiting queue");
        discharge(ps[2], 240);   // Kiran leaves ICU -> Harsh (emergency) takes the bed
        discharge(ps[3], 300);   // Meena leaves SURGICAL -> highest-priority waiting patient is
admitted

        System.out.println("\n[5] Resource utilisation (08:00-16:00)");
        System.out.printf("    Beds occupied: %d of %d%n", beds.occupied(), beds.total());
        for (String r : ors.rooms().keySet())
```

```
            System.out.printf("    %s utilisation: %.1f%%%n", r, ors.utilisation(r, 480));

        for (var e : waiting.entrySet())

            System.out.printf("    Still waiting for %s beds: %d%n", e.getKey(),
e.getValue().size());

    }

}
```

6.7 Complexity Analysis

Let P be the number of patients, W the number of patients waiting for a ward, R the number of operating rooms, B the number of bookings in one room and Bd the number of bookings of one doctor.

Phase	Operation	Time	Space
Admission	Insert P patients into the triage heap	O(P log P) in total	O(P)
Admission	Remove the next patient	O(log P)	O(1)
Bed management	Allocate or release a bed	O(1) average	O(number of beds)
Waiting	Add to or take from a ward queue	O(log W)	O(W)
Surgery	Overlap test for one room and doctor	O(log B + log Bd)	O(1)
Surgery	Book a fixed slot	O(log B + log Bd)	O(1) extra
Surgery	Find the earliest slot in all rooms	O(R·(B+Bd)·log(B+Bd))	O(B+Bd) candidates
Reporting	Utilisation of one room	O(B)	O(1)

Table 6.2: Time and space complexity of each operation

Justification. The heap has height O(log P), and insertion and removal move one element along one path, which gives O(log P). Beds are kept in a deque and two hash maps, so each operation is a constant number of average O(1) steps. The overlap test needs one floorEntry and one ceilingKey call on a red-black tree, each O(log B), and the same for the doctor’s tree. The slot search builds at most B + Bd candidate times for each of the R rooms, sorts them in a TreeSet in O((B+Bd) log(B+Bd)), and tests each candidate in O(log B + log Bd), so the total is O(R·(B+Bd)·log(B+Bd)). The earliest feasible start is always either the earliest allowed time or the end of an existing booking, so no better start is missed.

Scaling. A hospital has few operating rooms, so R is small, and the bookings of one day are limited, so B and Bd stay small. Triage and bed operations are logarithmic or constant and scale to many thousands of patients. The slot search is the most expensive operation, but it could be improved by walking only through the gaps between bookings or by using an interval tree. For a hospital with several departments, each department can run its own instance of the scheduler.

6.8 Verification

The modules were compiled and run on a test case of ten patients, eight beds and two operating rooms (Table 7.1), and every result was checked by hand. The emergency patients P3, P6 and P9 are served first, then the urgent patients P4, P7 and P10, then the routine patients P1, P2, P5 and P8. The ICU has two beds, so P9 has to wait. The surgical ward has three beds, which go to the urgent patients P4 and P10 and to P2, the routine patient who arrived first. The surgeries were checked against the timetable: Dr. Sen's operation on P4 cannot start before 10:30 because Dr. Sen is busy with P6 until then, and the 75-minute operation on P10 does not fit into the 30-minute gap in OR1, so it goes to OR2 at 10:30. The outputs are shown in Chapter 7.

7. RESULTS & DISCUSSION

The output of the running application and a short explanation of each result are given below. The test patients are listed in Table 7.1. The hospital has 8 beds (ICU 2, Surgical 3, General 3), 2 operating rooms (OR1, OR2) and 3 surgeons (Dr. Rao, Dr. Sen, Dr. Khan). The day starts at 08:00.

ID	Name	Priority	Ward	Surgery (min)	Doctor	Arrival no.
P1	Ravi	Routine	General	–	–	0
P2	Anita	Routine	Surgical	90	Dr. Rao	1
P3	Kiran	Emergency	ICU	120	Dr. Rao	2
P4	Meena	Urgent	Surgical	60	Dr. Sen	3
P5	Suresh	Routine	Surgical	45	Dr. Khan	4
P6	Lata	Emergency	ICU	150	Dr. Sen	5
P7	Farhan	Urgent	General	–	–	6
P8	Divya	Routine	Surgical	60	Dr. Rao	7
P9	Harsh	Emergency	ICU	–	–	8
P10	Neha	Urgent	Surgical	75	Dr. Khan	9

Table 7.1: Test patients used for the experiment

7.1 Console Output

```
[1] Admission in priority order (beds + surgeries, day starts 08:00)
    EMERGENCY   P3 Kiran   -> bed ICU-1
                           surgery 08:00-10:00 in OR1 with Dr. Rao
    EMERGENCY   P6 Lata    -> bed ICU-2
                           surgery 08:00-10:30 in OR2 with Dr. Sen
    EMERGENCY   P9 Harsh   -> no free ICU bed, added to waiting queue
    URGENT      P4 Meena   -> bed SURGICAL-1
                           surgery 10:30-11:30 in OR1 with Dr. Sen
    URGENT      P7 Farhan  -> bed GENERAL-1
    URGENT      P10 Neha   -> bed SURGICAL-2
                           surgery 10:30-11:45 in OR2 with Dr. Khan
    ROUTINE     P1 Ravi    -> bed GENERAL-2
    ROUTINE     P2 Anita   -> bed SURGICAL-3
                           surgery 11:30-13:00 in OR1 with Dr. Rao
    ROUTINE     P5 Suresh  -> no free SURGICAL bed, added to waiting queue
    ROUTINE     P8 Divya   -> no free SURGICAL bed, added to waiting queue

[2] Operating-room timetable
    OR1: 08:00-10:00 P3 (Dr. Rao)  10:30-11:30 P4 (Dr. Sen)  11:30-13:00 P2 (Dr. Rao)
    OR2: 08:00-10:30 P6 (Dr. Sen)  10:30-11:45 P10 (Dr. Khan)

[3] Conflict detection
    Book OR1 09:00-10:00 for Dr. Khan  -> REJECTED (room busy)
    Book OR2 12:00-13:00 for Dr. Rao   -> REJECTED (doctor busy)
    Book OR2 14:00-15:00 for Dr. Sen   -> ACCEPTED

[4] Discharges free beds for the waiting queue
    12:00: P3 Kiran discharged, bed ICU-1 freed
    EMERGENCY   P9 Harsh   -> bed ICU-1
    13:00: P4 Meena discharged, bed SURGICAL-1 freed
    ROUTINE     P5 Suresh  -> bed SURGICAL-1
```

```
                    surgery 13:00–13:45 in OR1 with Dr. Khan


[5] Resource utilisation (08:00–16:00)
    Beds occupied: 7 of 8
    OR1 utilisation: 65.6%
    OR2 utilisation: 59.4%
    Still waiting for ICU beds: 0
    Still waiting for SURGICAL beds: 1
```

Fig. 7.1: Output of the scheduling system

7.2 Explanation of the Results

Admission by priority. The three emergency patients P3, P6 and P9 are served first. The ICU has only two beds, so P3 and P6 get ICU-1 and ICU-2 and P9 goes to the ICU waiting queue. The urgent patients P4 and P10 take two of the three surgical beds, and the third goes to P2, the routine patient who arrived first. P5 and P8 wait for a surgical bed.

Surgery scheduling. P3 (Dr. Rao) and P6 (Dr. Sen) are operated first, in OR1 and OR2 from 08:00. P4 (Dr. Sen) must wait for the surgeon and starts at 10:30 in OR1. P10 (Dr. Khan) needs 75 minutes, which does not fit into the 30-minute gap in OR1, so it is placed in OR2 at 10:30. P2 (Dr. Rao) starts at 11:30 in OR1 when the room becomes free.

Conflict detection. A booking of OR1 from 09:00 to 10:00 is rejected because the room is occupied by P3, and a booking of OR2 from 12:00 to 13:00 for Dr. Rao is rejected because the room is free but the doctor is busy with P2 until 13:00. A booking of OR2 from 14:00 to 15:00 for Dr. Sen is accepted.

Discharge and waiting queue. When P3 is discharged at 12:00, ICU-1 is freed and the emergency patient P9 takes it at once. When P4 is discharged at 13:00, SURGICAL-1 is freed and P5 is admitted, because P5 arrived before P8, and the surgery of P5 is scheduled from 13:00 to 13:45 in OR1.

7.3 Summary of Results

Measure	Value
Patients processed	10
Beds / operating rooms / surgeons	8 (ICU 2, Surgical 3, General 3) / 2 / 3
Order of service	P3, P6, P9, P4, P7, P10, P1, P2, P5, P8
Waiting for a bed at first admission	3 (P9 for ICU, P5 and P8 for Surgical)
Booking attempts rejected	2 (one room clash, one doctor clash)
Waiting patients admitted after discharge	2 (P9 at 12:00, P5 at 13:00)
Beds occupied at the end	7 of 8 (87.5%)
OR1 utilisation (08:00-16:00)	65.6% (315 of 480 minutes)
OR2 utilisation (08:00-16:00)	59.4% (285 of 480 minutes)
Still waiting at the end	P8 (Surgical bed)

Table 7.2: Summary of results

7.4 Discussion

The results show that each part of the problem needs its own structure. The heap guarantees that emergencies are served before urgent and routine cases, and a first-come-first-served queue could not do this. The hash maps and queues make bed handling instantaneous. The ordered maps make the overlap test cheap and give the timetable in sorted order for free. Checking both the room and the doctor in one booking step removes the clashes that a manual schedule would only discover later.

The main limitations are that the test case is small, so no timing benchmark is claimed; that the data is held in memory only and is lost when the program ends; that a surgery duration is assumed to be known exactly; that an emergency patient cannot bump a surgery that is already booked; that cleaning time between operations is not modelled; and that there is no user interface.

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The Hospital Bed and Operating Room Scheduling System meets its objectives. It was built in Java without any external library and serves patients by priority, allocates beds by ward, schedules surgeries in the earliest slot where a room and a surgeon are both free, rejects double bookings, and gives freed beds to waiting patients. On the test case of ten patients, all emergency patients were served first, no clashes occurred, and the freed beds were reassigned in priority order, with 7 of 8 beds in use and the operating rooms 65.6% and 59.4% utilised.

The project also gave practical experience of the data structures studied in the course: a binary heap for priority, hashing and queues for beds, red-black trees for the timetables, and a greedy search for the earliest slot. The complexity analysis shows that admission and bed handling are logarithmic or constant, and that the slot search is the part that would need to be improved first for a very large hospital.

8.2 Future Scope

Near term

- Add cleaning and set-up time between surgeries, and a buffer after emergency operations.
- Allow an emergency patient to take the earliest slot by moving a routine surgery that has not started.
- Add more resource types, such as anaesthetists, nurses and equipment, to the booking check.

Medium term

- Speed up the slot search by walking through the gaps between bookings or by using an interval tree.
- Store patients, beds and bookings in a database so that the data survives a restart.
- Add a command-line or web interface for administrators, with timetable and occupancy views.

Long term

- Predict surgery duration and length of stay from past data to make schedules more reliable.
- Run one scheduler per department and coordinate them for hospital-wide bed management.
- Compare the greedy schedule with an optimal schedule from an integer-programming model.

9. REFERENCES

[1]T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

[2]B. Cardoen, E. Demeulemeester and J. Beliën, "Operating room planning and scheduling: A literature review," European Journal of Operational Research, vol. 201, no. 3, pp. 921–932, 2010.

[3]F. Guerriero and R. Guido, “Operational research in the management of the operating theatre: a survey,” Health Care Management Science, vol. 14, no. 1, pp. 89–114, 2011.

[4]P. J. H. Hulshof, N. Kortbeek, R. J. Boucherie, E. W. Hans and P. J. M. Bakker, "Taxonomic classification of planning decisions in health care: a structured review of the state of the art in OR/MS," Health Systems, vol. 1, no. 2, pp. 129–175, 2012.

[5]J. W. J. Williams, “Algorithm 232: Heapsort,” Communications of the ACM, vol. 7, no. 6, pp. 347–348, 1964.

[6]L. J. Guibas and R. Sedgewick, "A dichromatic framework for balanced trees," Proceedings of the 19th Annual Symposium on Foundations of Computer Science, pp. 8–21, 1978.

[7]J. Kleinberg and É. Tardos, Algorithm Design, Addison-Wesley, 2006.

[8]Oracle, Java Platform, Standard Edition Documentation: https://docs.oracle.com/en/java/javase/21/docs/api/